2026

Smart Pantry Manager



Monareng Marape

Richfield College

10/23/2026

Table of Contents

1.1 Introduction 3
1.1.1 Background 3
3.2 Problem Statement 3
3.3 Aim and Objectives 3
3.4 Target Users 3
3.5 Scope of the Application 3
1.2 System Design 4
1.2.1 System architecture 4
1.2.2 Screen-Flow Diagram 4
1.2.3 Database Design 5
1.2.4 Pantry Table 5
1.2.5 Recipes Table 5
1.2.6 Recipe Ingredients Table 6
1.2.7 Entity-Relationship (ER) Diagram 6
Figure 2: Entity-relationship diagram of the SQLite database. 7
1.2.8 Database Operations 7
1.3 Screenshots output 7
1.4 Key code snippets 12
1.4.1 SQLite Database Creation 12
1.4.2 Adding an ingredient 13
1.4.3 Retrieving Pantry Ingredients 14
1.4.4 Updating and Deleting Ingredients 14
1.5 Challenges and Solutions 15
1.5.1 Designing the User Interface 15
1.5.2 Storing and Managing Ingredient Data 15
1.5.3 Maintaining Functionality While Updating the Interface 15
1.6 Conclusion and Reflection 16
1.6.1 Conclusion 16
1.6.2 Reflection 16
References 17

Table of figures

Figure 1:Home page SmartPantry manager 8
Figure 2:Add ingredient screen 9
Figure 3:Suggested Recipes 10
Figure 4:Pantry screen 11
Figure 5:Settings screen 12

Table of tables

Table 1:Pantry Table 5
Table 2:Recipes Table 5
Table 3:Recipe ingredients table 6

GITHUB Link

https://github.com/Mona-reng/Smart-Pantry-Manager.git

1.1 Introduction

1.1.1 Background

Smart Pantry Manager is an Android mobile application developed to help users manage food ingredients stored in their pantry. People may sometimes forget which ingredients they have, how much is available, or when certain ingredients are due to expire. This can lead to food waste, unnecessary purchases and difficulty deciding what meals to prepare.

3.2 Problem Statement

The application addresses the problem of managing pantry ingredients manually. Without an organised system, users may struggle to keep track of available ingredients, update quantities, remove items they no longer have, and identify meals they can prepare using the ingredients available to them.

3.3 Aim and Objectives

The main aim of Smart Pantry Manager is to provide a simple mobile solution for organising pantry ingredients and helping users discover possible meals.

The objectives are to:

- Allow users to add ingredients with their names, quantities, units and optional expiry dates.
- Display saved ingredients in a pantry list.
- Allow existing ingredients to be edited or deleted.
- Suggest possible recipes based on ingredients stored in the pantry.
- Provide a simple home screen and settings screen for easy navigation and app information.

3.4 Target Users

The application is intended for individuals, students, families and other users who want to organise their food ingredients, reduce avoidable food waste and make better use of the food they already have.

3.5 Scope of the Application

Smart Pantry Manager includes ingredient management, pantry listing, basic recipe suggestions and an application settings screen. Recipe suggestions are based on the ingredient-matching functionality implemented in the application. The application is intended as a practical pantry-management tool rather than a complete meal-planning or online grocery-shopping system.

1.2 System Design

1.2.1 System architecture

Smart Pantry Manager is an Android mobile application developed using Java and XML layouts in Android Studio. SQLite is used as the local database to store pantry information and to define tables for recipes and recipe ingredients.

The application consists of five main screens:

- **Home screen:** Provides navigation to the main features.
- **My Pantry:** Displays saved ingredients and provides options to edit or delete them.
- **Add Ingredient:** Allows users to enter an ingredient's name, quantity, unit and optional expiry date.
- **Suggested Recipes:** Allows users to request meal suggestions based on available pantry ingredients.
- **Settings:** Displays application information and the version number.

The DatabaseHelper class extends Android's SQLiteOpenHelper class. It creates the database tables and provides methods to insert, retrieve, update and delete pantry ingredients.

1.2.2 Screen-Flow Diagram

The following diagram illustrates the navigation between the main screens of Smart Pantry Manager.

flowchart TD

A["Home Screen"] --> B["My Pantry"]

A --> C["Add Ingredient"]

A --> D["Suggested Recipes"]

A --> E["Settings"]

B --> F["Edit Ingredient"]

F --> B

B --> A

C --> G["Save Ingredient"]

G --> B

C --> A

D --> H["Find Recipes"]

H --> D

D --> A

E --> A

Figure 1: Screen-flow diagram for Smart Pantry Manager.

The home screen acts as the central navigation point. Users can open the pantry, add ingredients, view suggested recipes or access settings. The pantry screen supports ingredient editing and deletion. After adding an ingredient, users can return to the pantry to view their saved information.

1.2.3 Database Design

The application uses an SQLite database named smart_pantry.db, with the database version set to 1. The database helper defines three tables: pantry, recipes and recipe_ingredients.

1.2.4 Pantry Table

The pantry table stores information about the ingredients available to the user.

Table 1:Pantry Table

Field	Data type	Description
id	INTEGER	Primary key; automatically generated
name	TEXT	Ingredient name; required
quantity	REAL	Quantity available; required
unit	TEXT	Measurement unit; required
expiry_date	TEXT	Ingredient expiry date; optional

1.2.5 Recipes Table

Table 2:Recipes Table

Field	Data type	Description
id	INTEGER	Primary key; automatically generated
name	TEXT	Recipe name; required
instructions	TEXT	Recipe preparation instructions; required

1.2.6 Recipe Ingredients Table

Table 3:Recipe ingredients table

Field	Data type	Description
id	INTEGER	Primary key; automatically generated
recipe_id	INTEGER	References a recipe in the recipes table
ingredient_name	TEXT	Name of the required ingredient
quantity	REAL	Required ingredient quantity
unit	TEXT	Measurement unit

The recipe_id field is defined as a foreign key referencing recipes.id. The database definition specifies ON DELETE CASCADE, meaning associated recipe-ingredient records are intended to be deleted when their parent recipe is deleted, provided foreign-key enforcement is enabled.

1.2.7 Entity-Relationship (ER) Diagram

The following diagram represents the tables and the relationship defined in the database code.

```
erDiagram
  RECIPES ||--o{ RECIPE_INGREDIENTS : contains

  RECIPES {
    INTEGER id PK
    TEXT name
    TEXT instructions
  }

  RECIPE_INGREDIENTS {
    INTEGER id PK
    INTEGER recipe_id FK
    TEXT ingredient_name
    REAL quantity
    TEXT unit
```

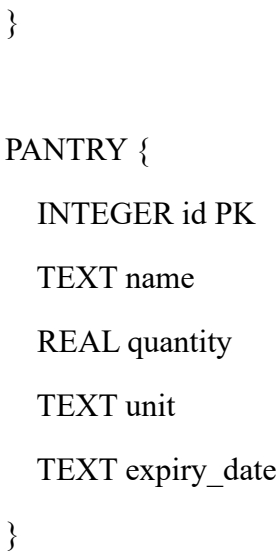


Figure 2: Entity-relationship diagram of the SQLite database.

The diagram shows a one-to-many relationship between recipes and recipe_ingredients: one recipe can have multiple ingredient records, while each recipe-ingredient record references one recipe. The pantry table is shown separately because the provided database code does not define a foreign-key relationship between it and the other tables.

1.2.8 Database Operations

The DatabaseHelper class implements four main pantry operations:

1. **Create:** addIngredient() inserts a new ingredient using ContentValues and returns whether the insertion succeeded.
2. **Read:** getAllIngredients() retrieves pantry records ordered alphabetically by ingredient name.
3. **Update:** updateIngredient() modifies an existing record using its ID.
4. **Delete:** deleteIngredient() removes a record using its ID.

These operations allow the application to perform the basic Create, Read, Update and Delete (CRUD) functions required for pantry management. Using a local SQLite database allows pantry records to be stored on the device between app sessions.

1.3 Screenshots output



Figure 1:Home page SmartPantry manager

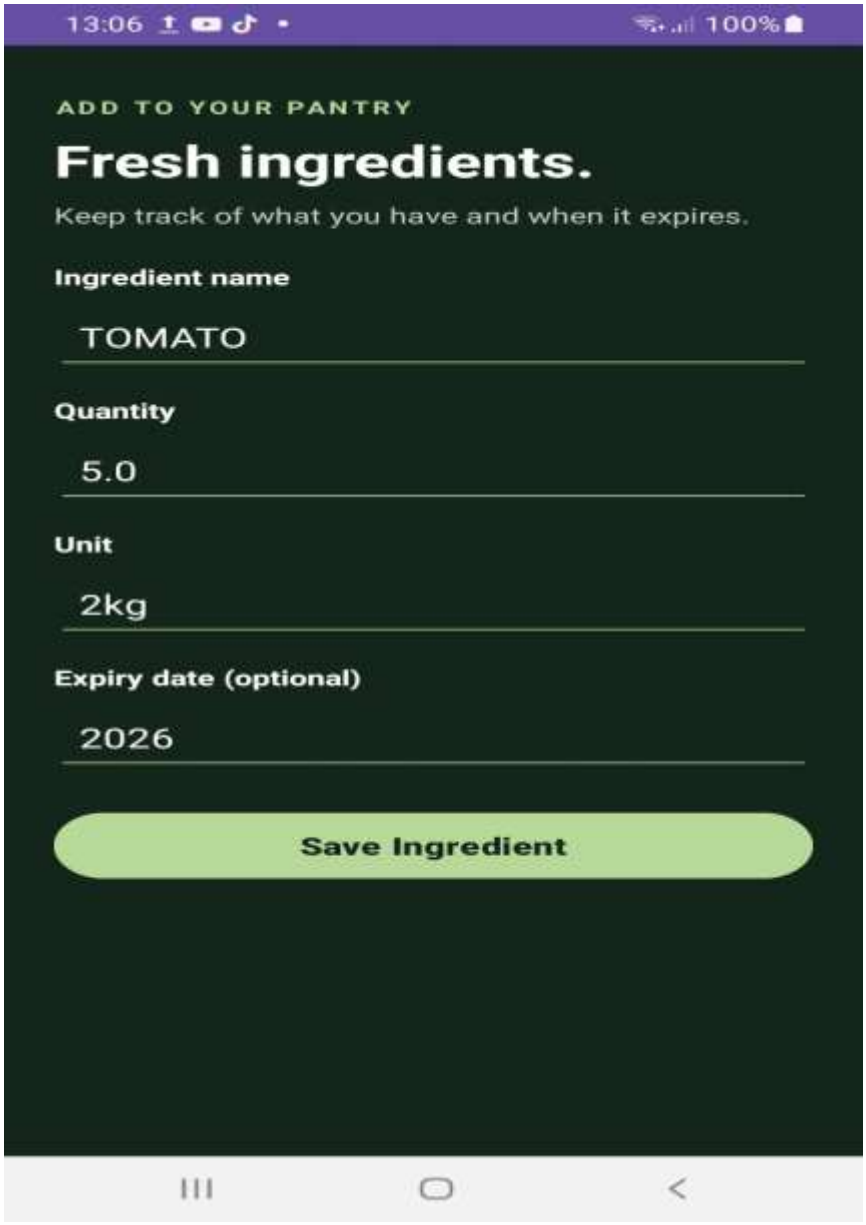


Figure 2:Add ingredient screen



Figure 3:Suggested Recipes



Figure 4:Pantry screen

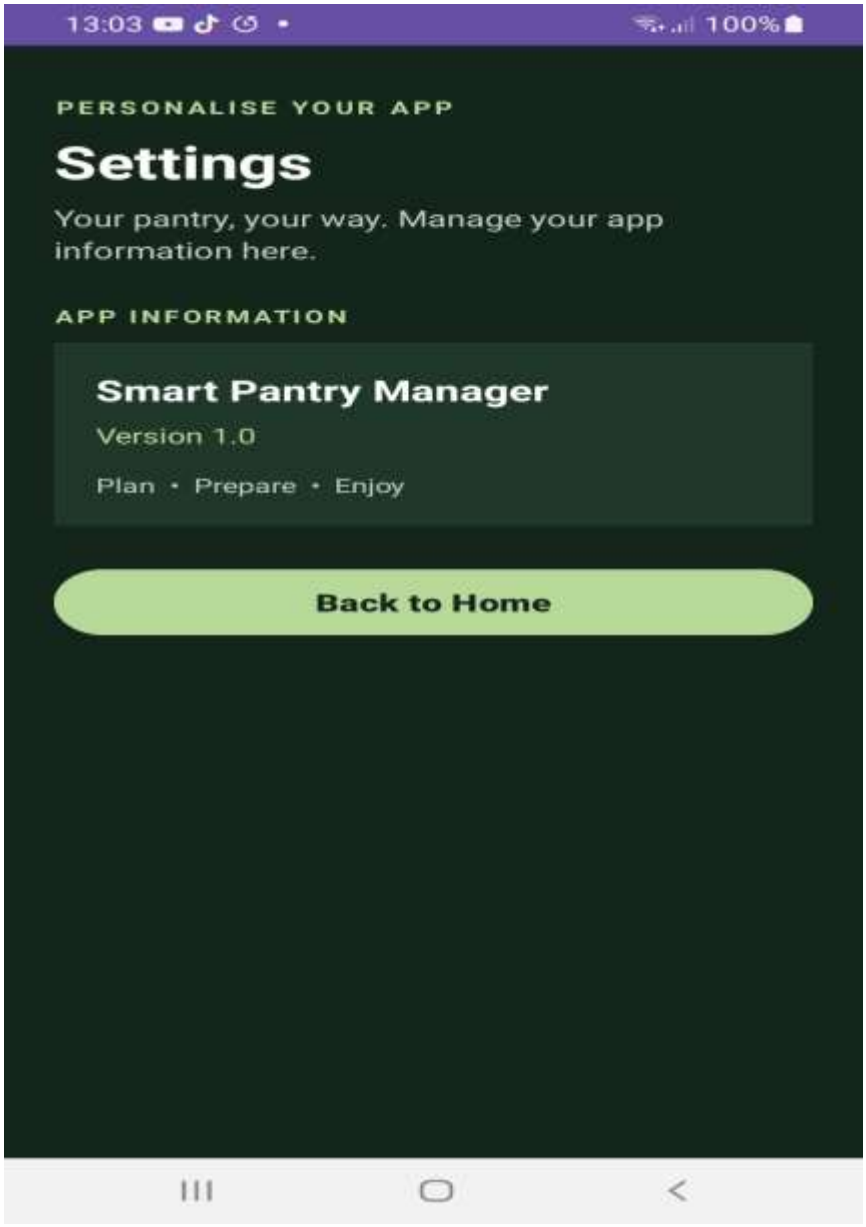


Figure 5:Settings screen

1.4 Key code snippets

1.4.1 SQLite Database Creation

The following code from DatabaseHelper.java creates the pantry table in the SQLite database.

```
String createPantryTable =
    "CREATE TABLE " + TABLE_PANTRY + " (" +
    COL_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
    COL_NAME + " TEXT NOT NULL, " +
```

```
        COL_QUANTITY + " REAL NOT NULL, " +
        COL_UNIT + " TEXT NOT NULL, " +
        COL_EXPIRY + " TEXT)";

db.execSQL(createPantryTable);
```

Explanation: This code defines the structure of the pantry table. The id field uniquely identifies each record, while the other fields store the ingredient name, quantity, unit and expiry date. The NOT NULL constraints ensure that the name, quantity and unit must be provided. The expiry date is optional.

1.4.2 Adding an ingredient

The addIngredient() method saves a new ingredient in the pantry table.

```
public boolean addIngredient(String name,
        double quantity, String unit,
        String expiryDate) {

    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues values = new ContentValues();
    values.put(COL_NAME, name.trim());
    values.put(COL_QUANTITY, quantity);
    values.put(COL_UNIT, unit.trim());
    values.put(COL_EXPIRY, expiryDate);

    long result = db.insert(TABLE_PANTRY, null, values);

    return result != -1;
}
```

Explanation: The method places the ingredient details into a ContentValues object and inserts them into the pantry table. It returns true if the insertion succeeds and false if it fails. This functionality allows users to add ingredients to their pantry.

1.4.3 Retrieving Pantry Ingredients

The getAllIngredients() method retrieves the ingredients stored in the database.

```
public Cursor getAllIngredients() {
    SQLiteDatabase db = this.getReadableDatabase();


    return db.query(
            TABLE_PANTRY,
            null,
            null,
            null,
            null,
            null,
            COL_NAME + " ASC"
    );
}
```

Explanation: This method reads the records from the pantry table and sorts them alphabetically by ingredient name. The returned Cursor allows the application to access the retrieved records and display them in the My Pantry screen.

1.4.4 Updating and Deleting Ingredients

The application allows users to edit ingredient details and remove ingredients that are no longer required.

```
public boolean deleteIngredient(int id) {
    SQLiteDatabase db = this.getWritableDatabase();


    int result = db.delete(
            TABLE_PANTRY,
            COL_ID + "=?",
            new String[]{String.valueOf(id)}
    );
```

```
    return result > 0;
}
```

Explanation: The deleteIngredient() method removes a record using its unique ID. The selection condition ensures that the deletion targets the selected ingredient rather than removing every record. The application also provides an updateIngredient() method to modify an existing ingredient's details.

These code snippets demonstrate the main database operations used by Smart Pantry Manager to create, retrieve, update and delete pantry records.

1.5 Challenges and Solutions

During the development of Smart Pantry Manager, several aspects of Android application development required attention. The following challenges should be retained in the report only if they accurately reflect the problems encountered during the project.

1.5.1 Designing the User Interface

Challenge: Creating a consistent and professional interface for the different screens required careful arrangement of layouts, text fields and buttons.

Solution: XML layout files were used to organise the interface elements. A consistent dark-green theme, light-green accents and readable text were applied across the main screens to improve the application's appearance and usability.

1.5.2 Storing and Managing Ingredient Data

Challenge: The application needed a reliable way to store ingredient details and make them available after users navigated between screens.

Solution: SQLite was implemented through the DatabaseHelper class. Database methods were created to insert, retrieve, update and delete ingredient records. This allowed the pantry screen to display saved information and supported the editing and deletion of ingredients.

1.5.3 Maintaining Functionality While Updating the Interface

Challenge: Updating the XML layouts required care to ensure that the interface changes did not interfere with the Java code responsible for the application's functionality.

Solution: The existing view IDs were preserved while the layouts were redesigned. The project was rebuilt after the changes to check for errors, and the application screens were tested to confirm that the main navigation and pantry functions continued to work.

These challenges provided practical experience in Android development, database management, XML layout design and testing.

1.6 Conclusion and Reflection

1.6.1 Conclusion

Smart Pantry Manager was developed as an Android application to help users manage the ingredients they keep at home. The application provides a home screen from which users can access the My Pantry, Add Ingredient, Suggested Recipes and Settings screens.

The pantry management functionality allows users to add ingredients, view saved records, edit ingredient details and delete records when they are no longer needed. SQLite provides local data storage, while the redesigned interface uses a consistent dark-green theme to create a clear and professional appearance.

The Suggested Recipes screen provides recipe suggestions based on selected ingredient names supported by the application's current implementation. Although the application offers basic pantry management and recipe suggestions, additional features could be introduced to make it more comprehensive.

Overall, the project demonstrates the practical use of Java, XML layouts, Android activities and SQLite database operations in the development of a mobile application.

1.6.2 Reflection

Developing Smart Pantry Manager provided an opportunity to apply programming concepts to a practical problem. The project helped develop an understanding of how Android activities, XML layouts and database operations work together to create a functional application.

The development process also highlighted the importance of testing the application after making changes. Preserving view IDs, checking build results and verifying that buttons perform their intended actions helped maintain the application's functionality.

In the future, the application could be improved by adding automatic expiry-date reminders, ingredient search and filtering, more detailed recipe recommendations, and a feature that suggests meals based on the ingredients available in the pantry. Data backup and synchronisation could also be considered.

This project strengthened practical programming and problem-solving skills and provided a foundation for developing more advanced Android applications.

References

Android Developers. (n.d.). *SQLiteOpenHelper*. Android Developers Documentation. Available at: https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper

[Accessed 23 October 2026].

Android Developers. (n.d.). *SQLiteDatabase*. Android Developers Documentation. Available at: https://developer.android.com/reference/android/database/sqlite/SQLiteDatabase

[Accessed 23 October 2026].

Android Developers. (n.d.). *Cursor*. Android Developers Documentation. Available at: https://developer.android.com/reference/android/database/Cursor [Accessed 23 October 2026].

Android Developers. (n.d.). *Layouts*. Android Developers Documentation. Available at: https://developer.android.com/develop/ui/views/layout/declaring-layout [Accessed 23 October 2026].

Android Developers. (n.d.). *Activities*. Android Developers Documentation. Available at: https://developer.android.com/guide/components/activities [Accessed 23 October 2026].